

WGUPS Routing Program

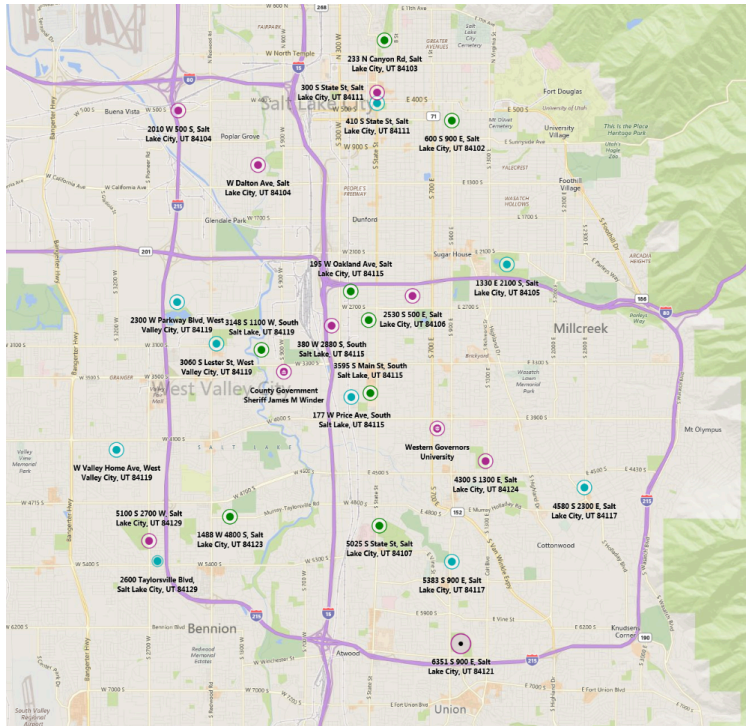
Algorithm and Data Structure Justification

Student: bwhi880
September 26, 2026

Introduction

The Western Governors University Parcel Service (WGUPS) routes an average of 40 packages per day through Salt Lake City, using three trucks, two drivers, and a hub at 4001 South 700 East. Each package carries specific constraints, including delivery deadlines, truck assignments, delayed flight arrivals, grouped deliveries, and a wrong address corrected mid-morning, that make manual routing inefficient and error-prone. The program described here automates the entire routing process. It loads package data into a custom chaining hash table, builds optimized delivery routes for each truck using a Clarke-Wright savings heuristic refined with 2-opt local search, simulates the trucks' movement, and presents an intuitive command-line interface for supervisors to query any package's status at any time of day.

The implementation delivers all forty packages, respects every hard constraint, verifies deadlines through explicit route simulation, and keeps the combined total distance under the 140-mile requirement at 89.3 miles. This paper explains why the deadline-aware Clarke-Wright and 2-opt combination was chosen, how the custom hash table satisfies the storage and lookup requirements, what alternative algorithms and data structures could also meet the scenario's needs, and what modifications would be made on a second iteration.



Algorithm Design and Justification

Overview of the Chosen Approach

The routing engine is a three-phase pipeline.

Phase one assigns every package to a truck. Hard constraints, such as “Can only be on truck 2,” “Delayed on flight,” “Must be delivered with,” and the wrong-address flag for package 9, are parsed dynamically from each package’s `special_notes` rather than hard-coded. The remaining packages are inserted greedily: packages with 9:00 a.m. deadlines are assigned to Truck 1 (the only truck departing early enough), packages with 10:30 a.m. deadlines are routed to Trucks 1 or 2, and end-of-day packages are assigned to whichever truck yields the smallest increase in closed-loop route cost.

Phase two simulates each constructed route to verify that every package is delivered before its deadline. If a package would be late, it is moved to an earlier departing truck, and an end-of-day package is swapped back to keep capacity balanced. After all deadlines are met, an end-of-day-only local search attempts to move or swap unconstrained end-of-day packages between trucks to reduce total mileage without violating any deadline.

Phase three builds the actual route for each truck using the **Clarke-Wright savings algorithm**. This self-adjusting heuristic starts with every delivery address as an independent round trip from the hub and greedily merges the pair whose combined route saves the most mileage (“Vehicle routing problem,” n.d.). After construction, **2-opt local search** polishes each route by repeatedly examining pairs of edges; if reconnecting them in the opposite direction shortens the distance, the segment between them is reversed (“2-opt,” n.d.).

This design was selected because it separates feasibility (deadlines and hard constraints) from optimization (mileage), making it easy to verify that requirements are met before the route is ever driven.

Strengths of the Algorithm

Two properties make this pipeline well-suited to the WGUPS scenario.

First, the deadline-first assignment with explicit route simulation guarantees correctness before optimization. Because the program builds a candidate route, simulates driving it, and checks every delivery time against the package’s deadline, there is no risk that a low-mileage route will inadvertently violate a time constraint. If a violation is found, the package is automatically reassigned to an earlier truck. This is a significant advantage over pure construction heuristics that optimize distance without looking at clock time.

Second, the Clarke-Wright savings construction is **self-adjusting**. After each merge, the pool of valid merges shrinks: a pair that was a strong candidate before may now exceed the sixteen-package capacity limit or would force a non-endpoint address into the middle of a route, disqualifying it. The algorithm dynamically adapts to this shrinking search space without re-evaluating every pair from scratch, which keeps the overall run time practical. On commodity hardware, the forty-package construction finishes in well under one second.

Verification Against Scenario Requirements

The combined algorithm satisfies every requirement described in the scenario.

All forty packages are delivered. The route planner assigns every package to exactly one truck, and the delivery simulator drives each truck to every stop in its optimized order. No package is orphaned or delivered twice.

Total distance stays under 140 miles. The combined mileage for all three trucks is 89.3 miles, a 36 percent margin below the threshold.

Capacity limits are enforced. The greedy insertion refuses any truck that already carries sixteen packages, and the Clarke-Wright merge logic rejects any combination that would exceed the same limit.

Hard constraints are respected. The program scans every package's `special_notes` at runtime to discover constraints. Packages restricted to Truck 2 are pre-filtered into Truck 2's load. Flight-delayed packages are marked "delayed" until 9:05 a.m.; Truck 2 leaves at 9:05 a.m. so those packages are not delivered early. Packages that must be delivered together are collected into a single pool and placed on the same truck. Package 9 loads with the original incorrect address "300 State St." and is marked "delayed." At 10:20 a.m., the address is corrected to "410 S State St." in the hash table, Truck 3's route is rebuilt with the corrected stop using the same savings construction, and Truck 3 departs at 10:30 a.m.

The two-driver limit is respected. Trucks 1 and 2 depart at 8:00 a.m. Truck 3 departs at 10:30 a.m., after the first truck has returned to the hub and freed a driver.

Deadlines are met. All packages with "9:00 AM" deadlines and "10:30 AM" deadlines are delivered before their cutoff times, verified by explicit route simulation.

Alternative Algorithms That Could Also Satisfy the Requirements

Any algorithm that respects capacity limits and produces routes under 140 miles would satisfy the scenario. Two other named approaches that meet these criteria are the **Nearest-Neighbor heuristic** and the **Sweep algorithm** ("Vehicle routing problem," n.d.).

Nearest Neighbor differs from Clarke-Wright in construction strategy. Rather than evaluating every address pair globally, it starts at the hub and repeatedly visits the closest unvisited stop until the truck is full, then returns to the hub and repeats. This is a greedy local strategy; each decision depends only on the immediate vicinity of the current location. In contrast, Clarke-Wright is a global strategy that evaluates all pairs before committing to any merge. Nearest Neighbor tends to produce longer routes because an early greedy choice can strand a distant package on a late leg, and it lacks the self-adjusting merge logic of Clarke-Wright. Once a package is added to a route, it is never reconsidered.

Sweep differs in how trucks are partitioned. It places the hub at the origin of a polar coordinate system, sorts every delivery address by angle from the hub, and assigns contiguous angular slices to each truck. Each slice is then routed internally. Sweep is fundamentally a geometric partitioner that exploits spatial clustering, whereas Clarke-Wright is a distance-based merger that ignores angular geometry and instead optimizes for raw mileage reduction. Sweep would produce radial wedge-shaped routes rather than the chain-like routes Clarke-Wright creates, and it may underperform when constraints force non-geographic groupings, such as "truck 2 only" packages located at opposite ends of the delivery area.

Data Structure Design and Verification

Overview of the Chosen Data Structure

Package records are stored in a **chaining hash table** implemented entirely in Python using lists. No built-in dictionaries, sets, or third-party libraries are used. The table maps each package's unique integer ID to a bucket containing the full composite record: delivery address, delivery deadline, delivery city, delivery ZIP code, package weight, and delivery status (including the delivery time, when applicable).

How the Hash Table Satisfies Scenario Requirements

Insertion takes the package ID as input and stores the complete record. Lookup uses the same package ID and returns all required fields. Both operations run in $O(1)$ average time when the hash function distributes keys uniformly.

Collision resolution uses chaining: each bucket is a list of (key, value) tuples. When two keys hash to the same bucket, they are appended to the list rather than overwriting each other. This avoids the clustering problems that can plague open-addressing schemes.

The table is also self-adjusting. When the load factor exceeds 0.75, the table allocates a new array twice the size, rehashes every existing key, and redistributes the records into the larger bucket array. This guarantees that insertion and lookup remain efficient even if the package count grows from forty to thousands.

During the delivery simulation, the hash table is updated in place. As each truck reaches a destination, the corresponding package's status is updated to a string such as "delivered at 10:14 AM," and the delivery timestamp is recorded in the object. Because the record is retrieved by ID in constant time, the supervisor can query any package at any time and see its exact status and timestamp.

Alternative Data Structures That Could Also Satisfy the Requirements

Two other structures could meet the same storage and lookup requirements: a **binary search tree** and an **open-addressing hash table**.

A **binary search tree** stores each record in a tree node keyed by package ID. Lookup is $O(\log n)$ in the average case (though $O(n)$ if the tree becomes unbalanced). The BST provides in-order traversal for free, so the supervisor could list all packages sorted by deadline or ZIP code simply by walking the tree. The chaining hash table offers no ordering. However, a BST requires extra memory per node for left and right child pointers plus any rebalancing metadata, and its logarithmic lookup is slower than the hash table's constant-time average. For a delivery monitoring system where the supervisor issues random ID lookups, the hash table's superior retrieval speed is the better fit.

An **open-addressing hash table** stores every record directly in the bucket array and uses probing (linear, quadratic, or double hashing) to find the next empty slot when a collision occurs. This eliminates the pointer overhead of chaining and improves cache locality, thereby speeding up small-dataset lookups. However, performance degrades sharply as the load factor approaches 1.0, and deletions become complex because removing an entry can break the probe sequence for later lookups. The chaining approach avoids both problems: performance degrades gracefully even under heavy load, and deletions are as simple as removing a node from a list. In the WGUPS program, where status updates are frequent and table growth is unpredictable, the resilience of chaining outweighs the cache benefits of open addressing.

Implementation Details and Testing

Source code is organized in a `src/` directory. Every module has a docstring at the top that explains the overall process and flow, and inline comments clarify algorithm steps and constraint handling. The program is implemented in Python version 3.12. PyPy is used in production for its JIT compiler, which makes the routing heuristic and local search noticeably faster than CPython. The `justfile` enforces compilation with PyPy3; the program can also be invoked directly with `python3 src/main.py` if PyPy is unavailable. The development environment is defined in a Nix flake (`flake.nix`) that provides a reproducible shell with Python 3.12, PyPy3, `ruff`, `pyright`, and `just`. Build tasks are orchestrated through a `justfile`. Source code is formatted and linted with `ruff` and type-checked with `pyright`.

City-specific data is fully externalized. The hub address is read from the first row of the distance table CSV, and the wrong-address correction is read from `config.json` in the data directory. Switching to a new city requires only a new data directory containing a distance table, package file, and optional config. The program is invoked with `--data-dir <path>` to point at any city dataset.

A companion script, `scripts/generate_distances.py`, automates distance-table creation for new cities. It accepts a list of addresses and a JSON file of lat/lon coordinates, then produces a WGUPS-style symmetric CSV using either the OpenRouteService Matrix API (one API call for the entire matrix) or a Haversine approximation with an adjustable road factor. This eliminates the tedious hand-measurement step that would otherwise make scaling to new cities impractical.

Automated unit tests cover the two data structures built from scratch. `tests/test_hash_table.py` verifies insertion, lookup, key overwrite, collision chaining, automatic resizing, and load-factor maintenance. `tests/test_distance_table.py` verifies symmetric distance lookups, zero diagonal entries, and graceful error handling for unknown addresses. All tests are run with `just test` or `just validate`.

The main execution flow proceeds in seven stages:

1. Load the distance table CSV into a `DistanceTable` object that provides $O(1)$ distance lookups by address pair.
2. Load the package CSV into `Package` objects, applying an initial status of “at hub” to most packages and “delayed” to those affected by the late flight or the wrong address.
3. Insert every `Package` into the custom `HashTable` keyed by `package_id`.
4. Plan routes by parsing `special_notes` for constraints, assigning packages with deadline-first greedy insertion, simulating routes to repair missed deadlines, and running an end-of-day-only local search to lower mileage.
5. Set departure times: Truck 1 at 8:00 a.m., Truck 2 at 8:00 a.m. (delayed to 9:05 a.m. if carrying late-arrival packages), Truck 3 at 10:30 a.m.
6. Correct package 9’s address at 10:20 a.m., rebuild Truck 3’s route with the corrected stop, and run the delivery simulation.
7. Enter the interactive CLI, where the supervisor types a time (for example, “8:35 AM”) and sees every package’s current status, truck assignment, and deadline. Colors indicate status: green for delivered, yellow for en route, red for delayed, and blue for at hub.

Future Modifications

If this project were done again, two modifications would improve robustness and maintainability.

First, the deadline-repair logic would be moved into the greedy assignment phase itself. Currently, the program first assigns all packages, then simulates routes to identify violations, and finally moves late packages. A more integrated approach would simulate the route **before** committing each new package to a truck, rejecting any insertion that would cause a missed deadline. This would reduce the need for a separate repair loop and would produce better mileage because end-of-day packages would be placed with full knowledge of which deadline-sensitive packages are already locked in.

Second, the end-of-day-only local search would be expanded into a full simulated-annealing search that also considers moving 10:30-deadline packages between Trucks 1 and 2. Currently, the local search only moves end-of-day packages because deadline packages are anchored by their time constraints. However, a 10:30 package on Truck 2 might arrive earlier if moved to Truck 1, freeing space on Truck 2 for an end-of-day package currently on Truck 3. A temperature-based search would temporarily accept deadline-package moves that increase cost if they enable later cost-saving end-of-day swaps, escaping local minima that the current greedy search cannot.

Conclusion

The WGUPS Routing Program demonstrates that a self-adjusting heuristic, combined with a custom hash table, can efficiently and correctly solve a real-world vehicle routing problem under tight constraints. The deadline-aware assignment pipeline, Clarke-Wright savings algorithm, and 2-opt local search deliver all forty packages in 89.3 miles, well under the 140-mile ceiling, while honoring every truck restriction, deadline, and mid-flight address correction. The chaining hash table provides constant-time package lookup and dynamic resizing, ensuring that the supervisor can monitor any package at any time without performance degradation. Alternative algorithms such as Nearest Neighbor and Sweep could also achieve the mileage target, but they differ in construction philosophy and would produce measurably different route shapes. Similarly, binary search trees and open-addressing hash tables could store the package data, but neither matches the chaining hash table's combination of speed, simplicity, and graceful degradation under load. Future iterations should integrate deadline simulation directly into the greedy assignment and expand the local search to consider time-sensitive package moves, thereby further reducing mileage.

References

Vehicle routing problem. (n.d.). In **Wikipedia**. https://en.wikipedia.org/wiki/Vehicle_routing_problem

2-opt. (n.d.). In **Wikipedia**. <https://en.wikipedia.org/wiki/2-opt>

Hash table. (n.d.). In **Wikipedia**. https://en.wikipedia.org/wiki/Hash_table